



AP

Assignment 3

Anders Hoeck, Marcus Frehr Friis-Hansen

September 2026

1 Introduction

In this assignment we extended the Parser for APL via. implementation of function application, operators, syntax for print, put and get, lambdas, let-binding, loops and try-catch. The implementation is functional, compiles without errors and passes all tests. During implementation we used Chatgpt and Claude, to reason about code, fix syntax errors and bugs, design tests etc. We also used them to reason about the numbered questions, e.g, we used them to check and correct the grammar syntax for Question 1.

2 Numbered Questions

2.1

Show the final and complete grammar with left-recursion removed and all ambiguities resolved

```
Atom ::= int | bool | var | "(" Exp ")"

FExp ::= Atom { Atom }

LExp ::= FExp
        | "if" Exp "then" Exp "else" Exp
        | "print" string Atom
        | "get" Atom
        | "put" Atom Atom
        | "\" var ">" Exp
        | "let" var "=" Exp "in" Exp
        | "try" Exp "catch" Exp
        | "loop" var "=" Exp "for" var "<" Exp "do" Exp

Exp2 ::= LExp [ "*" Exp2 ]

Exp1 ::= Exp2 { ("*" | "/" ) Exp2 }

Exp0 ::= Exp1 { ("+" | "-" ) Exp1 }

Exp  ::= Exp0 { "==" Exp0 }
```

2.2

Why might or might it not be problematic for your implementation that the grammar has operators `*` and `**` where one is a prefix of the other

Since `*` is a prefix of `**`, a parser could potentially view the first `*` as a multiplication operator when it should view `**` as the power operator. In our implementation, this is not problematic because `**` is parsed in **pExp2**, which is called before `*` is parsed in **pExp1**. This means `**` is recognized as the power operator before the parser handles `*`.

2.3

Where in your parser is backtracking (try) strictly necessary, and where is it merely convenient? For one try you could remove, state what changes: the language recognised, the error messages, or the

running time?

Backtracking (try) is strictly necessary in **lKeyword** since keywords can be prefixes of valid variable names. For example, **getx** must be parsed as the variable **getx**, rather than failing after trying to parse **get** as a keyword.

Backtracking (try) is merely convenient in **pLExp** around **if**, in the case of incomplete or broken **If** statements. For example in the case of any broken input starting with "if", **if a then b**, which is missing "else", (try) would attempt to but also fail at all other branches in **pLExp**, since no other branch begins with "if", and finally produce a generic error message. Without (try) the failure would immediately be reported at the point of failure and give a more specific error message.

2.4

Give an example of where in your parser it matters that choice is ordered (picks the first successful parse) (or equivalently for < | >, if you have used that instead of choice). If no such case occurs, explain why

Ordered choice matters in **pLExp**. The parser tries the expressions forms (if, print, get, put ect.) before the fallback **pFExp**. For example, when parsing *if x then y else z*, the **if** alternative succeeds first, so the parser produces an if expression. The order is therefore important because **pFExp** is the fallback after more specific expression forms.

2.5

Suppose APL is extended with comments notation (such as line comments or block comments as in Haskell). How would you implement this in your parser, and would it require many modifications?

We would implement a parser for comments and include it together with whitespace. We would replace the use of space in lexeme with this shared space/comment parser. Since most of the lexical parsers (**lVName**, **lInteger**, **lKeyword**, **lString** ect.) already use lexeme, comments would automatically be allowed between tokens. So only a small number of modifications would be needed for this implementation.